INDIAN PEDIATRICS
AI ASSISTANT

STEP 1

Set Up the Development Foundation

A beginner's implementation chapter: make the project reproducible, journal-neutral, safe to extend, and easy to check before acquiring content.

Teaching manual installment 1.0 | October 2026

Project: Reusable medical-journal literature assistant
POC corpus: Indian Pediatrics, January 2025, Volume 62, Issue 1
Audience: AI students and developers learning to plan and build a grounded literature assistant

How to use this installment
These are the first chapters of a cumulative teaching manual. They explain decisions made so far and provide a reproducible way to understand them. The full 300+ page reference is intended to grow as later roadmap phases are completed.

Document control and alignment

This chapter follows **Phase 1 - Development Foundation** in the roadmap and the project-approved Step 1 label, **Set Up the Development Foundation**. The legacy Step 1 PDF is used as a learning reference, but the actual implementation here uses the current Project folder, current Python runtime, journal TOML files, and a lightweight foundation rather than claiming every legacy API/UI item was delivered.

Target workspace: **D:\AAAA Pediatric Assistant\AI Asst for IP\Project**. This path is authoritative for current work. The folder is nested under the supplied AI Asst for IP directory.

> **Step 1 outcome**
> A new developer can identify the project root, create or use an isolated Python environment, load a journal-specific configuration through shared code, run a local startup check, and verify the foundation using one repeatable checker.

Contents at a glance

- 1. Purpose, prerequisites, and acceptance criteria
- 2. Decisions: stack, modularity, and journal reuse
- 3. Project layout and the role of each file
- 4. Reproduce the environment on Windows
- 5. Configuration and journal separation
- 6. Startup behavior and safe secret handling
- 7. Step 1 readiness checkpoint
- 8. Troubleshooting, teaching exercises, and handoff

1. Purpose, prerequisites, and acceptance criteria

Step 1 turns a product idea into an executable software workspace. It establishes names, boundaries, configuration, reproducibility, and a basic program entry point. Its output is deliberately boring: a clean start and clear project organization. That boring foundation lowers the cost of every later change.

This phase does not acquire or process article content, create embeddings, build a vector database, call an LLM, or implement the final UI. Those belong to later phases. Avoid reporting a successful startup as proof that RAG works.

1.1 Learner outcomes

- Explain why a virtual environment isolates project packages.
- Locate shared application logic separately from journal-specific settings.
- Run configuration loading and the startup smoke check.
- Explain why an example environment file is different from a private .env file.
- Read a readiness report and distinguish failure from warning.
- Recognize the difference between a template that is reusable and one that merely has a generic name.

1.2 Step 1 acceptance criteria

Criterion	How the project checks it	Interpretation
Python version	Checker confirms >= 3.11.	Python 3.12 is current development target; a bundled runtime is not a portable project requirement.
Project environment	Checker verifies interpreter runs from Project/.venv.	Avoid installing dependencies globally or into another project.
Reusable structure	Required config, data, source, scripts, tests and evaluation directories exist.	Structure reserves clear homes for future phases.
Foundation files	README, pyproject, requirements, .env.example and config files exist.	A second developer can understand and reproduce the foundation.
Configuration	Indian Pediatrics TOML loads and identifies issue 766.	This is configuration validation, not a permission grant.
Git root	The repository root is the Project folder.	Version history should apply to the application workspace.
Startup smoke check	Local entry point prints READY and exits successfully.	Tests that imports/config/start path work; not an AI answer test.
Warnings visible	Unconfirmed full-text terms are reported as warnings.	Warnings are tracked for appropriate later decisions.

2. Decisions: stack, modularity, and journal reuse

Choose the smallest set of tools that supports the current phase. Technology choices are provisional architecture decisions; they should be revisited when requirements or evidence change.

Choice	Why it fits now	Why not add more now
Python 3.11+ (3.12 target)	Strong ecosystem for document processing, search, AI APIs and Windows development.	The foundation currently needs only standard-library capabilities; no package pile is required.
PowerShell for Windows instructions	Native shell for the target platform and existing user environment.	Avoid relying on Unix-only commands.
Git at Project root	Change history and rollback for source and configuration.	A remote GitHub setup is not a prerequisite unless explicitly approved and configured.
TOML configuration	Readable, standard-library parsing, suitable for journal/app settings.	Avoid duplicating a setting in source code and config.
Standard-library startup module	Keeps Phase 1 install simple, reproducible and quick to check.	No need yet for FastAPI, Streamlit, LangChain, Chroma, pandas, PDF libraries or model SDKs.
Shared src package + per-journal config	Separates generic workflow code from each journal's identity and URLs.	Do not fork the whole codebase for each journal.
Local file data folders	Keep raw, processed, indexes and cache distinct; make provenance visible.	Do not place source PDFs in Python module folders.

2.1 Why defer the LLM, embedding, and UI connections?

At this step there is no defined prompt, evidence retrieval behavior, or user workflow to test. Installing AI packages or configuring paid API credentials now increases setup complexity without validating journal retrieval. The project keeps `.env.example` and configuration paths ready for later, but its current readiness checker expects no third-party packages for the foundation.

2.2 Reuse model

The shared package belongs in `src/journal_assistant/`; journal-specific facts belong in `config/journals/.toml`. The example template demonstrates the fields expected for another journal. Where websites differ substantially, later work should implement a separate source adapter rather than adding journal-specific assumptions throughout shared code.

3. Project layout and the role of each file

The actual structure is designed to grow in phases. Empty folders with a `.gitkeep` are intentional: Git does not retain empty directories, but the folders communicate planned boundaries.

Path	Purpose now	Future use / guardrail
config/app.toml	Application-wide defaults.	Shared settings; avoid journal-specific facts here.
config/journals/indian_pediatrics.toml	Journal identity, URLs, selected issue, access review status.	One file per authorized journal; keep rights status explicit.
config/journals/journal_template.toml.example	Reusable configuration template.	Copy and complete for another journal; do not leave placeholder values in an active config.
src/journal_assistant/	Shared configuration loader and startup logic.	Reusable application code; keep user interface and journal data separate.
scripts/run_app.py	Local foundation entry point.	Later scripts for ingestion and checks can be added here.
scripts/check_step1.py	Automated readiness checks for Step 1.	Keep checks repeatable, clear, and low-risk.
scripts/check_step2.py and scripts/prepare_step2_metadata.py	Step 2 corpus catalogue and checkpoint support.	Step-specific utilities; later add migration/version handling if schema changes.
data/raw/	Original PDFs supplied for the POC.	Treat as immutable; verify provenance; ignored by Git by default.
data/metadata/	Article catalogue, manifest, checkpoint notes.	Structured records are the bridge between files and retrieved evidence.
data/processed/	Reserved for derived text and cleaned documents.	Never overwrite the originals here.
data/indexes/ and data/cache/	Reserved for generated search artefacts.	Usually reproducible and excluded from source control.
tests/ and evaluation/	Reserved for software tests and answer/retrieval evaluation.	Do not confuse code tests with evidence-quality evaluation.
.env.example / .gitignore	Variable names and ignored local/secret/generated files.	Never store live keys in the example file or commit a local .env.

3.1 Generic journal onboarding checklist

- Choose a stable journal ID (lowercase underscore or hyphen form).
- Copy the journal template; add display name, website, issue-index URL, selected issue metadata, and access-review status.
- Record site structure and section terms in a source note; do not assume the source has Indian Pediatrics' structure.
- Verify each source URL and a small authorized document sample before adapting parsers.
- Run the foundation check with the new config selected, then add journal-specific acquisition checks later.
- Keep shared code journal-neutral; record any necessary source-specific parsing adapter as an explicit module.

4. Reproduce the environment on Windows

Use Windows PowerShell opened in the Project root. The README is the current source of setup commands. A machine-specific bundled Python runtime was used to create the current environment; students should use their own supported Python installation rather than copying that machine path.

From the Project folder:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python --version
python -m pip --version
python -m pip install -r requirements.txt
```

The current Step 1 `requirements.txt` contains no third-party dependencies. This is intentional. The activation command changes the active shell environment; the checker independently validates that Python is executing from the project's `.venv`. If activation scripts are disabled by local policy, follow the organization's approved PowerShell approach rather than bypassing security settings.

4.1 Dependency discipline

- Add a package only when a phase needs it; record it in the project dependency declaration.
- Use the project's active interpreter to install packages so they do not land in an unrelated Python.
- When adding runtime packages, pin or constrain versions according to the team's reproducibility policy and verify imports.
- Do not copy `pip freeze` wholesale without review if it captures unrelated environment packages.
- Never put credentials in install commands, scripts, notebooks, screenshots or documentation.

5. Configuration and journal separation

Configuration is data that controls behavior without changing reusable code. TOML is used because it is readable and parsed by Python's standard library. The loader validates expected fields and makes an invalid config fail visibly rather than letting a later operation use wrong source settings.

Configuration area	Example fields	Validation question
Application	shared defaults and selected config pointer	Are defaults independent of a specific journal?
Journal identity	journal_id, display_name, homepage and issue URLs	Do URLs lead to the intended publisher and index?
Initial corpus	title, issue_id, date, volume, issue number	Does this match the selected issue listing?
Access review	review_status and full_text_permission	Does it report only what has actually been reviewed?
Example template	placeholder identity and expected fields	Can a second journal owner fill this in without editing source code?

5.1 Secrets are not journal metadata

The `.env.example` file is a safe template and may contain names such as an API variable without a real secret value. A private `.env` is local and must be excluded from Git. Journal source URLs and permission statuses belong in journal configuration; API tokens belong in private environment variables or a managed secret store. Never conflate having a working API key with permission to process copyrighted journal text.

6. Startup behavior and safe secret handling

The foundation runner loads configuration and returns a clear readiness indication. It is deliberately not a chatbot: no model call, embedding request, retrieval, or PDF extraction is implied by the word `READY`. The small entry point gives later modules a stable place from which to grow.

6.1 What a smoke check proves

- The Python interpreter is usable.
- The expected configuration file can be found, parsed, and validated.
- The package imports and local startup code execute without an exception.
- The Project root and required foundation files can be identified.

6.2 What it does not prove

- That the source website is reachable now.
- That the PDFs are permission-cleared for every use.
- That extraction preserves article meaning.
- That a vector database or language model works.
- That answers are medically accurate, clinically safe, or validated.

7. Step 1 readiness checkpoint

Run the checker from the active Project virtual environment. Its pass/fail output is the evidence for the foundation gate; keep the final output with the implementation notes when repeating setup on a new computer.

```
python scripts\check_step1.py
```

Checker group	What it checks	How to respond
Runtime	Supported Python and active Project .venv.	Activate the correct environment or recreate it.
Structure and required files	Expected directories and foundation files exist.	Create the missing item from project instructions; do not work around by weakening the check.
Journal configuration	The selected Indian Pediatrics config loads and names issue 766.	Correct TOML or choose the intended configuration file.
Project metadata and dependencies	TOML metadata is valid and Step 1 requirements are intentionally empty.	Keep dependency list phase-appropriate.
Git root	Git repository top directory equals Project.	Initialize or repair repository from the intended root.
Startup smoke test	Entry point exits successfully and prints READY.	Inspect captured error output and fix the cause.
Access-review warning	Unconfirmed full-text status is visible.	Track this honestly in corpus collection and external-use decisions.

7.1 Recorded result for this project

The Step 1 checkpoint previously reported **9 passed, 0 failed, 1 warning**. The warning was the journal configuration's pending confirmation for full-text access/indexing. It was a warning rather than a foundation failure: Step 1 establishes software readiness, while content rights and corpus scope are handled explicitly in Step 2. The user later supplied local files and stated access for the internal POC; that statement is recorded as the acquisition basis, not as a general publisher licence.

8. Troubleshooting, teaching exercises, and handoff

Troubleshooting should start from the first failing line, not from reinstalling every package. Use the checker as a diagnostic map, read its specific label, and repair the underlying cause.

Symptom	Likely cause	First check
Python version fails	Older or unexpected Python is on PATH.	Run `python --version` and `Get-Command python`; create `.venv` using intended interpreter.
Virtual environment fails	Shell is using global Python or another project's env.	Activate `.venv` and inspect `python -c "import sys; print(sys.prefix)"`.
TOML fails to load	Syntax error, wrong path, or required value absent.	Inspect exact TOML file, quotes, sections, and configured path.
Git root is wrong	Repository was initialized in parent or nested directory.	Inspect `git rev-parse --show-toplevel`; do not silently add the parent workspace.
Startup does not print READY	Import/config error or unexpected app state.	Run the runner directly and read full traceback before changing code.
Access warning appears	Permission field still unconfirmed.	Keep warning visible and record the user's stated basis and restrictions; do not change status to approved without evidence.

8.1 Student exercises

- Draw the dependency graph from PowerShell and virtual environment to configuration loader and startup check.
- Create a second journal TOML by copying the example. Identify which fields are generic and which are journal-specific.
- Intentionally break a configuration key in a disposable copy and observe the check failure; restore it and rerun.
- Explain in your own words why a green startup check cannot establish clinical accuracy.
- Propose a new readiness assertion and state what failure it should catch without overclaiming.

8.2 Handoff to Phase 2

Before Phase 2, the workspace should be stable and the selected journal configuration should identify the intended issue. Phase 2 defines the corpus and acquires or inventories source documents. Do not begin chunking or embeddings merely because PDFs have been placed in `data/raw/`; first catalogue them and record role, source, acquisition status, and exclusions.

References and glossary

- Ver 1 - Implementation - Step 1.pdf (legacy detailed setup reference; its old path/runtime and API tests are not current project facts).
- Current Project README.md, pyproject.toml, requirements.txt, config/journals/indian_pediatrics.toml, scripts/check_step1.py and scripts/run_app.py.
- Ver 1 - Complete Implementation Guide for POC.pdf, Phase 1 Development Foundation.

Glossary

Term	Plain-language meaning
Virtual environment	A project-specific Python installation area for isolating packages.
Configuration	Settings stored outside reusable program logic.
Smoke check	A short run that verifies basic startup, not full correctness.
Journal adapter	A module handling source-specific structure while keeping shared workflows reusable.
Secret	Credential or token that must not be committed or exposed.
Git repository root	The top-level directory whose files are tracked by that repository.